

Project Proposal and Methodology

Hybrid RAG & Fine-Tuning for Intelligent Customer Support

1. Executive Summary

This project designs and evaluates a progressive AI pipeline for automating customer support response generation. Starting from a zero-shot baseline, the system evolves through two solution stages: a Naive Retrieval-Augmented Generation (RAG) system and a Hybrid RAG system powered by a LoRA fine-tuned intent router. The goal is to demonstrate that grounding LLM responses in authoritative policy documents — and further improving retrieval precision with a fine-tuned router — produces measurably more accurate, factual, and policy-compliant answers than a standalone language model.

2. Problem Statement

Large Language Models (LLMs) deployed for customer support suffer from two critical failure modes:

- **Hallucination:** The model generates plausible but factually incorrect responses, such as inventing delivery timelines or non-existent policies.
- **Lack of grounding:** Without access to company-specific SOPs (Standard Operating Procedures), responses are generic and fail to reflect actual business rules.

These failures erode customer trust and create compliance risks. The solution is to build a system where the LLM is constrained to answer using only verified policy documents, and where the retrieval step is guided by a model that understands customer intent.

3. Project Objectives

1. Establish a zero-shot baseline using a pre-trained instruction-tuned LLM and quantify its hallucination rate.
2. Build a Naive RAG pipeline that retrieves relevant SOP content before generating a response.
3. Fine-tune the base model with LoRA to act as an intent router, improving retrieval precision.
4. Evaluate all three architectures on a held-out test set using ROUGE, BLEU, format adherence, and intent accuracy.

5. Produce a final comparative analysis demonstrating the incremental value of each component.

4. Dataset

4.1 Primary Dataset — Bitext Customer Support

Property	Value
Source	Hugging Face (bitext/Bitext-customer-support-llm-chatbot-training-dataset)
Original size	26,872 rows
Sampled size	~4,000 rows (stratified)
After deduplication	3,902 rows
Columns used	instruction, intent, category, response
Intent classes	27 unique intents
Categories	11 categories (CONTACT, ORDER, PAYMENT, etc.)

Deduplication was performed using normalised comparison (lowercase + strip) to catch near-identical samples. Rows with null values in instruction, intent, or category were dropped.

4.2 Knowledge Base — SOP Documents

Property	Value
Format	Markdown (.md)
Total files	13 SOP documents
Topics covered	Order tracking, cancellations, refunds, shipping, payments, password reset, technical troubleshooting, escalation matrix, and more

These documents serve as the factual ground truth for all RAG-based responses. They are chunked (chunk size = 500, overlap = 50) and embedded into a persistent ChromaDB vector store.

5. Data Preparation

5.1 Instruction Formatting

Each dataset row is converted into ChatML format using the Qwen2.5 tokenizer's `apply_chat_template`. The assistant's target response is a strict JSON string containing the intent and category:

```
<|im_start|>system
You are Qwen, created by Alibaba Cloud. You are a helpful assistant.<|im_end|>
<|im_start|>user
I can't track my order<|im_end|>
<|im_start|>assistant
{"intent": "track_order", "category": "DELIVERY"}<|im_end|>
```

This structured target trains the model to emit machine-parseable intent classifications.

5.2 Tokenisation

- **Tokenizer:** Qwen/Qwen2.5-1.5B-Instruct
- **Max length:** 256 tokens
- **Padding:** Right-padded to max length
- **Label masking:** Padding tokens replaced with -100 (PyTorch ignore index), so the loss is not computed on padding

5.3 Dataset Splits

Split	Size	Percentage
Train	3,121	80%
Validation	390	10%
Test	391	10%

Splits are **stratified by intent** to ensure all 27 intent classes are represented in every partition. Zero leakage was verified programmatically using set-intersection checks on content hashes.

The test set (`df_test.csv`) is held out completely and used only for final evaluation in Notebooks 5 and 7.

6. System Architecture

The project implements three progressively improved architectures:

Baseline → Naive RAG (V1) → Hybrid RAG (V2)

6.1 Stage 1 — Baseline

Architecture: Pre-trained LLM, zero-shot inference, no external context.

Model: Qwen/Qwen2.5-1.5B-Instruct loaded in 4-bit quantisation (NF4, bfloat16 compute) using BitsAndBytes.

Process: 1. Customer query is passed directly to the model via a ChatML prompt 2. Model generates a response from parametric knowledge only 3. No retrieval, no policy grounding

Expected failure mode: Model hallucinates delivery timelines, invents policies, or deflects with generic AI disclaimers.

6.2 Stage 2 — Naive RAG (Solution V1)

Architecture: Dense retrieval + LLM generation.

Components: - **Embedding model:** sentence-transformers/all-MiniLM-L6-v2 - **Vector store:** ChromaDB (persistent) - **LLM:** Qwen/Qwen2.5-1.5B-Instruct

Process: 1. Customer query is embedded using the sentence transformer 2. Top-1 SOP chunk is retrieved from ChromaDB via cosine similarity 3. Retrieved SOP and query are combined into an augmented prompt 4. Model generates a response grounded in the retrieved policy

Limitation: The raw customer query (e.g. “I want to cancel”) may not semantically match the SOP header (e.g. “# Order Cancellation Policy”). Retrieval can be imprecise when query phrasing differs from document structure.

6.3 Stage 3 — Hybrid RAG (Solution V2)

Architecture: Fine-tuned intent router + dense retrieval + LLM generation.

New component: LoRA-fine-tuned Qwen/Qwen2.5-1.5B-Instruct acting as an intent classifier.

Process: 1. The fine-tuned router receives the customer query and predicts a structured JSON intent (e.g. {"intent": "cancel_order"}) 2. The predicted intent string is used as the retrieval query (more precise than the raw noisy customer message) 3. Top-1 SOP chunk is retrieved from ChromaDB 4. Model generates a response grounded in the retrieved policy

Key insight: Using a clean, normalised intent keyword for retrieval (e.g. cancel_order) instead of the raw query (e.g. “i want 2 cancl my ordr pls”) produces more relevant SOP chunks and therefore more accurate answers.

7. Fine-Tuning Methodology

7.1 Approach — LoRA (Low-Rank Adaptation)

Full fine-tuning of a 1.5B parameter model is prohibitively expensive on consumer hardware. LoRA injects small trainable rank-decomposition matrices into selected layers, leaving the base model frozen. This reduces trainable parameters from ~1.5B to ~3.5M while preserving the model’s pre-trained capabilities.

7.2 LoRA Configuration

Parameter	Value
Rank (r)	16
Alpha	32
Target modules	q_proj, v_proj
Dropout	0.05
Bias	none
Task type	CAUSAL_LM

7.3 Training Configuration

Parameter	Value
Base model	Qwen/Qwen2.5-1.5B-Instruct
Quantisation	4-bit NF4 (bfloat16 compute)
Optimiser	AdamW
Learning rate	2e-4
Batch size	4 (train), 4 (validation)
Max steps	200
Validation frequency	Every 20 steps
Early stopping patience	5 validations
Training data	tokenized_train/ (3,121 samples)
Validation data	tokenized_valid/ (390 samples)

7.4 Training Outcome

The model was trained for 200 steps (~25% of one epoch). Validation loss showed a consistent downward trend, indicating the model was learning the intent classification task. Early stopping did not trigger within the 200-step budget. A longer training run would be expected to improve intent accuracy further.

The best checkpoint (lowest validation loss) was saved to `intent_lora_best/` and loaded for inference via `PeftModel.merge_and_unload()`.

8. Evaluation Framework

All three architectures are evaluated on the same 391-row held-out test set to ensure a fair comparison.

8.1 Metrics

Metric	Description
ROUGE-1	Unigram overlap between generated answer and reference
ROUGE-L	Longest common subsequence overlap
BLEU	N-gram precision of generated text vs reference
Format Adherence (%)	Percentage of responses that are non-empty strings
Exact Match (%)	Percentage of predicted intents matching ground truth exactly
Fuzzy Match (%)	Percentage matching with ≥ 80 fuzzy ratio (catches minor label variations)
Hallucination Frequency (%)	Proxy: percentage of responses with ROUGE-L < 0.20

8.2 Evaluation Design

- **Baseline:** Zero-shot response scored against ground truth response column
- **Naive RAG (V1):** SOP-retrieved response scored against ground truth response column
- **Hybrid RAG (V2):** Intent-routed, SOP-retrieved response scored; intent accuracy measured against ground truth intent column
- **Adversarial subset:** Filtered subset of test samples containing sentiment words (still, never, terrible, frustrated) to test robustness on emotionally charged queries

9. Technical Stack

Component	Technology
Language	Python 3.12
LLM framework	HuggingFace Transformers 4.55.0
Fine-tuning	PEFT 0.17.1 (LoRA)
Quantisation	BitsAndBytes 0.50.0

Component	Technology
Vector store	ChromaDB
Embeddings	sentence-transformers/all-MiniLM-L6-v2
Data processing	Pandas, HuggingFace Datasets
Evaluation	rouge-score, sacrebleu, rapidfuzz
Compute	Google Colab (T4 GPU, 16GB VRAM)

10. Expected Outcomes

Architecture	Expected Trend
Baseline	Low ROUGE/BLEU; high hallucination rate
Naive RAG (V1)	Improved ROUGE/BLEU vs baseline; reduced hallucinations
Hybrid RAG (V2)	Best intent accuracy; further improvement in response relevance with full training

The incremental improvement from Baseline → V1 isolates the contribution of **retrieval**. The further improvement from V1 → V2 isolates the contribution of **fine-tuned intent routing**.

11. Limitations and Future Work

- Training budget:** The fine-tuned router was trained for only 200 steps (~25% of one epoch) due to compute constraints. A full training run of 800+ steps would significantly improve intent accuracy.
- Reference quality:** ROUGE/BLEU scores are computed against the conversational response column from the Bitext dataset, which is not always SOP-aligned. SOP-grounded references would produce more meaningful scores.
- Single retrieval:** Only the top-1 SOP chunk is retrieved. Top-k retrieval with re-ranking could improve coverage for complex queries.
- Model scale:** Qwen2.5-1.5B is compact for deployment efficiency. Larger models (7B+) would improve baseline quality and reduce the gap that retrieval needs to close.